- *Don't use* `[BindProperties]`. In addition to the `[BindProperty]` attribute, there is a `[BindProperties]` attribute (note the different spelling) that can be applied to the Razor Page `PageModel` directly. This will cause all properties in your model to be model-bound, which can leave you open to overposting attacks if you're not careful. I suggest you don't use the `[BindProperties]` attribute and stick to binding a *single* property with `[BindProperty]` instead.
- *Accept route parameters in the page handler*. For simple route parameters, such as the `id` passed into the `OnGet` and `OnPost` handlers in listing 16.8, I add parameters to the page handler method itself. This avoids the clunky `SupportsGet=true` syntax for `GET` requests.
- *Always validate before using data*. I said it before, so I'll say it again: validate user input!

That concludes this look at model binding in Razor Pages. You saw how the ASP.NET Core framework uses model binding to simplify the process of extracting values from a request and turning them into normal .NET objects you can work with quickly. The most important aspect of this chapter is the focus on validation. This is a common concern for all web applications, and the use of `DataAnnotations` can make it easy to add validation to your models.

In the next chapter we'll continue our journey through Razor Pages by looking at how to create views. In particular, you'll learn how to generate HTML in response to a request using the Razor templating engine.